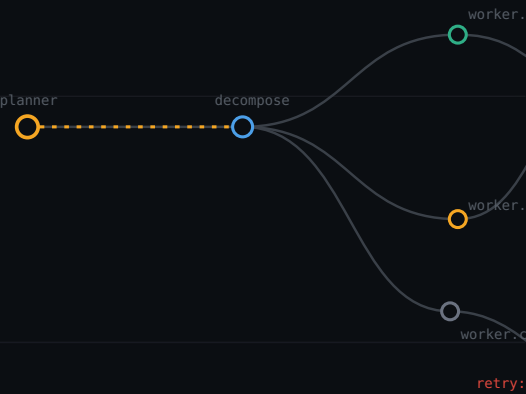


The Orchestration Surface

A visual direction for Veridex's operator console — turning a distributed multi-agent backend into something a person can watch, trust, and steer.

PREPARED FOR SCOPE BASELINE
Veridex core teamFrontend, phase 2Backend-complete, UI unstarted



The engine runs. No one can see the dashboard yet.

Reading the current build against what's still open, the imbalance is worth naming plainly: the backend is the hard part, and it's mostly done. The frontend is a connective shell. This document is about closing that gap on purpose, before the natural pull of "just wire the API to some cards" fills it by accident.

●built

What the platform can already do

- Planner/Worker decomposition over Celery + Redis queues
- Hard multi-tenancy — Organization → Workspace → Project, RBAC
- Six production connectors: GitHub, Gmail, Jira, Notion, Slack, Filesystem
- Qdrant vector pipeline, Neo4j knowledge graph, MinIO object storage
- LiteLLM provider abstraction, now routed to Gemini 2.5 Pro / Flash
- Dockerized, Istio-fronted, deployed end-to-end via `dep'loy.sh`

●unstarted

What has no surface yet

- A live view into what any agent is actually doing right now
- Any UI for the OPA-style YAML policies that govern access
- A place to read the DAG trace or an eval judgment after the fact
- Connector state — is the Slack sync healthy, stale, or broken
- Anything that turns "distributed" into "legible," for a human

The risk isn't a missing feature — it's a missing metaphor. Every enterprise tool eventually gets tables, filters, and a settings page. What's specific to an agent orchestrator is that its core subject — asynchronous, sometimes-failing, multi-step machine work — has no natural UI precedent the way inbox or CRM software does. Get that one metaphor right early, and every screen after it gets easier to design. Get it wrong, and every screen after it needs a workaround.

Make the trace the interface. Not a feature inside it.

Most agent-platform UIs treat "traces" as a debug panel bolted onto a dashboard that otherwise looks like every other admin console — KPI tiles up top, a table below, a settings gear in the corner.

Veridex already records the thing that actually matters: a DAG trace of what the Planner decided and what every Worker did with it. The proposal here is to stop hiding that behind a tab, and use it as the organizing visual grammar for the whole product.

Three commitments this implies

- 01 Nodes and edges everywhere, not just in the DAG viewer.** Governance policies, connector health, and eval results all get expressed as the same visual unit — a status-carrying node — so the whole platform reads as one connected system instead of four separate tools sharing a nav bar.
- 02 Color is reserved for state, never for decoration.** Amber means running. Mint means done. Red means blocked, and appears nowhere else. An operator should be able to squint at any screen and read system health before reading a single label.
- 03 The system speaks in its own true names.** Where the backend has a real term of art — queued, retrying, judged, committed — the UI uses that exact word. No softened synonyms invented for the sake of a friendlier tone.

THE UNIT, EVERYWHERE



`worker.summarize_thread`
`gemini-2.5-flash · 4.2s`

running



`policy.workspace_export_check`
`OPA rule · allow`

passed



`connector.slack_incremental`

blocked

Same card shape for an agent step, a policy check, and a connector sync. Same three places to look: what it is, what it's doing, and whether that's okay.

The token system

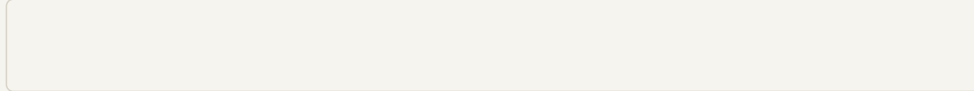
Everything downstream — every screen in this document, and every screen your team builds after it — derives from this palette, this type scale, and one spacing unit. Nothing here is arbitrary; each choice is explained against the job it does.

Color — status is the only thing color is allowed to mean



ink
#0B0E12

Base surface, structure



paper
#F6F4EF

Reading surface



cyan
#4C9FE8

Trace lines, structural data



amber
#F5A623

Live · running · attention



mint
#2FAE86

Committed · passed



red
#E0483D

Blocked only. Never elsewhere.

Type — three faces, three jobs

Display · Poppins

Headlines, section titles. Restrained — never body copy.

Body · Liberation Sans

Reading text, descriptions, all long-form content.

Mono · DejaVu Sans Mono

System-truth text only: IDs, statuses, YAML, queue names — the tell that it's not editorial copy.

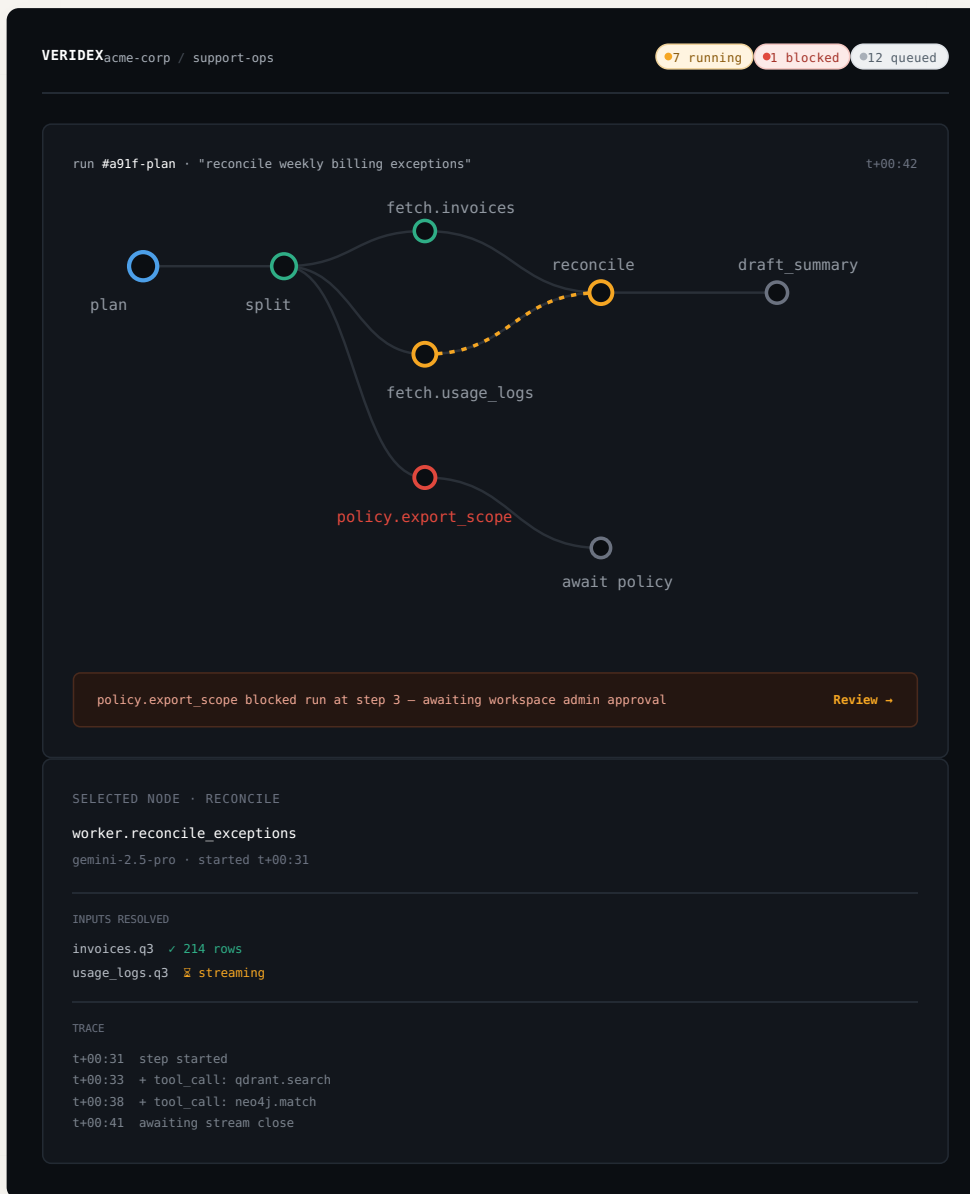
Spacing & elevation

One base unit, 4px, scaled geometrically (4 · 8 · 12 · 16 · 24 · 32 · 48). Two elevation levels only: flat cards on paper for structured data, and the ink surface reserved for "the system's own view" — trace inspector, policy engine, raw output. When the screen turns dark, you're looking at ground truth, not a summary of it.



The Orchestration Console

The room an operator actually lives in: every in-flight Planner run, laid out as a live tree of its Worker steps. This is where the trace-as-interface idea gets its fullest expression — the DAG isn't a diagram *about* the run, it is the run.



Reading order, deliberately: status counts first (top right), then the shape of the run, then one node's detail on demand. Nothing here is invented — `qdrant.search` and `neo4j.match` are literal calls this stack already makes; the console just makes them visible as they happen.

Governance & Policy

The OPA-style YAML policies and the RBAC models already exist at the database layer. What's missing is anywhere a workspace admin can read them, see what they've actually blocked, and check the immutable audit ledger — without opening a terminal.

policies/export_scope.yaml active

```

rule: export_scope
applies_to: [connector.slack, connector.gmail]
effect: require_approval
when:
  data_class: customer_pii
  destination: external
approvers:
  role: workspace_admin
  min_count: 1

```

Bound to 3 workspaces [View diff history](#)

audit ledger — immutable

blocked

run #a91f-plan hit export_scope
t+00:41 · pending approval from J. Alvarez

granted

Role connector_operator added to M. Osei
Yesterday, 4:12 PM · by workspace admin

policy edit

export_scope.yaml min_count raised 1 → 2
Jul 18 · by S. Chen · [view diff](#)

Why the ledger reads like a trace, not a log file. Same card anatomy as the console: a status pill, a plain-language line, a system-truth detail underneath. An admin who's never seen this screen before should still be able to read it, because it obeys the same three rules as everything else in the product.

Connector Health

Six connectors are already production-grade at the code level — GitHub, Gmail, Jira, Notion, Slack, Filesystem. None of them have anywhere to show whether a sync is current, stalling, or has quietly been failing for two days.

● GitHub

Incremental sync · 4 repos · last event 2 min ago

● healthy t-00:02

● Notion

Full sync complete · 1,204 pages indexed

● healthy t-00:11

● Jira

Incremental sync running · webhook lag rising

● degraded t-00:34

● Slack

OAuth token expired — [reconnect workspace](#)

● broken t-2d 03:00

● Gmail

Incremental sync · 3 mailboxes · last event 6 min ago

● healthy t-00:06

● Filesystem

Idle — no changes detected since last full sync

● idle t-06:40

Four states, exactly: healthy, degraded, broken, idle. "Degraded" exists on purpose — it's the state that gets skipped when a team only builds for up/down, and it's usually the one an operator needed the earliest warning about. The Slack row shows the pattern for every failure state in the product: name the cause in plain language, then give the one action that fixes it.

Evaluation & Trace Inspector

The LLM-as-Judge framework and the full DAG trace recording already produce this data — the gap is a reader for it. This screen sits on the ink surface deliberately: it is the one place in the product that shows the system's own record, unsummarized.

trace #a91f-plan · judged retrospectively eval complete

GROUNDNESS

0.94

TASK COMPLETION

1.0

POLICY COMPLIANCE

flagged

JUDGE RATIONALE

Output correctly reconciled 214 of 214 invoice rows against usage logs. Step `fetch.usage logs` retrieved a customer-identifying field before the export-scope policy check ran – flagged for review, not for output quality.

FULL TRACE, VERSIONED PROMPT INCLUDED

t+00:00	planner	prompt v14	decomposed into 4 steps
t+00:04	worker.a	tool_call	qdrant.search(query="q3 usage")
t+00:31	worker.reconcile	prompt v9	gemini-2.5-pro, 1,204 tokens
t+00:41	policy	export_scope → require_approval	
t+00:42	judge	groundedness 0.94, completion 1.0	

Versioned prompts show up here as plain facts — prompt v14, prompt v9 — because a team debugging a regression needs to know instantly whether the prompt changed underneath a run, not go looking for it.

Component grammar

The pieces every screen in this document is built from. Build these once, correctly, and every future screen — connectors ten and eleven, a fifth agent type — inherits the same visual language for free.

Status pill — five states, fixed vocabulary



Never invent a sixth. "Degraded" on the connector screen borrows running's amber rather than adding a color — the palette stays closed.

Node card — the one repeating unit



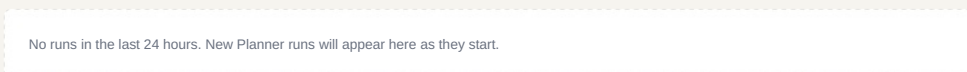
Icon-slot, mono identifier, one pill, right-aligned. This exact anatomy is reused for agent steps, policy checks, and connector syncs — see page 03.

Trace line — connective tissue, not decoration



Solid = settled. Dashed = actively resolving right now. This is the only place motion belongs in the whole product — everywhere else, state changes by color, not animation.

Empty & error states — the system's voice, not an apology



States what's true and what happens next. Never "Oops!" — never blames the person for a system condition.

Sequencing this against the next phase

The status report already names three candidate next phases. Here's how the console in this document lines up with them — and which order actually unblocks the others fastest.

●start here

Frontend UI/UX — build the Console first, not the whole app

The report frames this as general "UI/UX enhancement." Narrow it: the Orchestration Console (page 05) is the one screen that needs the DAG trace data Celery is already emitting. Building it first means the visual language in this document gets proven against real running data immediately, instead of against three mocked-up screens that all need retrofitting later.

●second

Governance enforcement — ship the YAML engine and its screen together

The report lists policy-engine middleware as its own phase, separate from UI. Don't split them: a policy that can block a run but has no screen (page 06) is invisible to the person it just blocked. Building the enforcement middleware and the Governance screen in the same milestone means every policy ships with a place to see it work.

●third

Knowledge chunking & embedding — Connector Health becomes the readiness check

Before wiring Connectors into Qdrant and Neo4j embedding jobs, the Connector Health screen (page 07) is what tells the team whether a given connector's sync is trustworthy enough to embed from yet. Ship the screen slightly ahead of the pipeline work, and it becomes the pre-flight check rather than an afterthought bolted on once something's already broken silently.

Nine screens, one grammar. Everything after this reuses it.

The backend already speaks in plans, workers, policies, and traces. This direction just gives that language a surface — so the tenth connector and the fifth agent type look like they always belonged, instead of needing a new pattern invented for them.

